

Formation Chef de Projet IA

Fondamentaux du Machine Learning

Thomas Walter Chloé-Agathe Azencott Florian Massip

Center for Computational Biology (CBIO)
Mines Paris – Institut Curie – INSERM U900
PSL Research University & PR[AI]RIE, Paris, France

November 26, 2025

Programme

- Introduction à l'apprentissage automatique
- **Cours 1** : Minimisation du risque empirique
- **Cours 2** : Régularisation et sélection de modèle
- **Cours 3** : Arbres et méthodes ensemblistes
- **Cours 4** : Méthodes à noyaux
- **Cours 5** : Réduction de dimension
- **Cours 6** : Clustering
- Travaux pratiques : mise en œuvre des notions et algorithmes avec scikit-learn (Python numérique).

1. Évaluation d'un modèle

1.1 Généralisation et surapprentissage

Défi principal de l'apprentissage supervisé :

- Il est relativement facile d'entraîner un modèle qui « marche » bien (faible erreur de prédiction) sur les données d'apprentissage
Exemple extrême : apprentissage « par cœur »

1.1 Généralisation et surapprentissage

Défi principal de l'apprentissage supervisé :

- Il est relativement facile d'entraîner un modèle qui « marche » bien (faible erreur de prédiction) sur les données d'apprentissage
Exemple extrême : apprentissage « par cœur »
- **Généralisation** : capacité du modèle à **faire de bonnes prédictions sur des données dont on ne connaît pas l'étiquette**

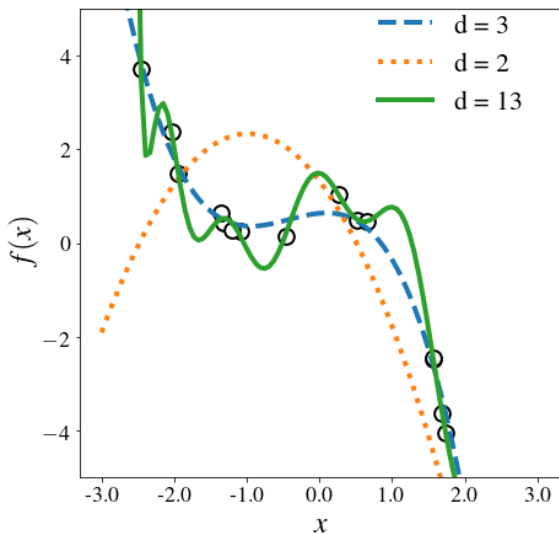
1.1 Généralisation et surapprentissage

Défi principal de l'apprentissage supervisé :

- Il est relativement facile d'entraîner un modèle qui « marche » bien (faible erreur de prédiction) sur les données d'apprentissage
Exemple extrême : apprentissage « par cœur »
- **Généralisation** : capacité du modèle à **faire de bonnes prédictions sur des données dont on ne connaît pas l'étiquette**
- **Surapprentissage** : quand la performance est meilleure sur les données d'apprentissage que sur de nouvelles données (overfitting)

Surapprentissage

Simulation : vrai modèle = polynôme de degré 3 + bruit.



Compromis biais-variance

1.2 Jeux d'entraînement et de test

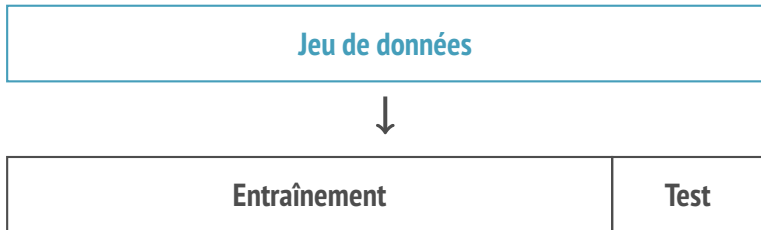
- **Erreur de généralisation** : erreur que l'on peut attendre sur de nouvelles données (la définition formelle fait appel à l'espérance)

1.2 Jeux d'entraînement et de test

- **Erreur de généralisation** : erreur que l'on peut attendre sur de nouvelles données (la définition formelle fait appel à l'espérance)
- Elle est **estimée** en **mettant de côté** une partie des données :
 - On sépare les données en un **jeu d'entraînement** et un **jeu de test** (typiquement, 80%–20%)

1.2 Jeux d'entraînement et de test

- **Erreur de généralisation** : erreur que l'on peut attendre sur de nouvelles données (la définition formelle fait appel à l'espérance)
- Elle est **estimée** en **mettant de côté** une partie des données :
- On sépare les données en un **jeu d'entraînement** et un **jeu de test** (typiquement, 80%–20%)



- Le **jeu d'entraînement** (train set) sert à apprendre le modèle
- Le **jeu de test** sert à estimer l'erreur de généralisation du modèle

Règle d'or

NE PAS TOUCHER au jeu de test
sauf pour évaluer l'erreur de généralisation du modèle

1.3 Validation croisée

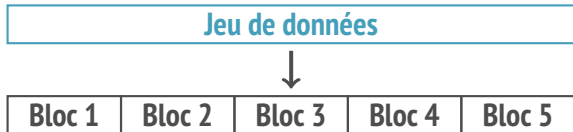
- La **validation croisée** (cross-validation) permet :
 - d'utiliser toutes les données pour l'entraînement et pour la validation
 - d'obtenir une performance moyenne (+/- écart-type) moins sensible au choix du jeu de test
 - On sépare le jeu de données en K **blocs** (folds)

En pratique, K=5 ou K=10 le plus souvent (équilibre entre le nombre d'expériences et la taille de chaque jeu d'entraînement)
 - On utilise tour à tour chacun des blocs comme **jeu de validation** et l'union des autres comme **jeu d'entraînement**
- ⇒ K scores de performance → **performance de généralisation** du modèle

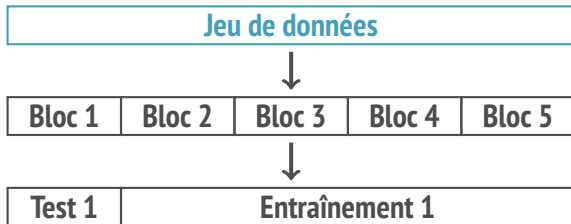
1.3 Validation croisée

Jeu de données

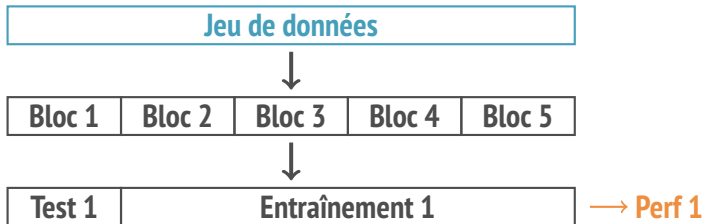
1.3 Validation croisée



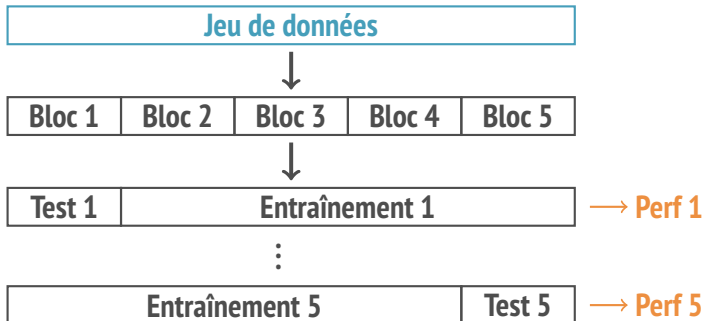
1.3 Validation croisée



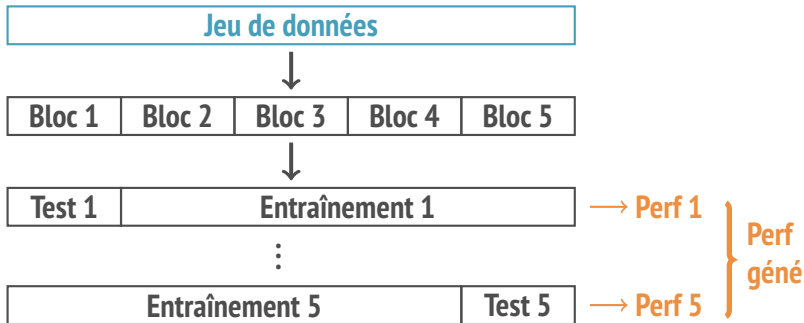
1.3 Validation croisée



1.3 Validation croisée



1.3 Validation croisée



1.4 Évaluation d'un modèle (classification)

- **Pourcentage d'erreur :** proportion d'observations mal classifiées

1.4 Évaluation d'un modèle (classification)

- **Pourcentage d'erreur** : proportion d'observations mal classifiées
- **Problème** : cas où les classes ne sont **pas équilibrées**
 - Exemple** : détection de fraude
 - 99% des observations ne sont pas des fraudes
 - Un modèle qui prédit toujours « non » a un pourcentage d'erreur de 1%.

1.4 Évaluation d'un modèle (classification)

- **Pourcentage d'erreur** : proportion d'observations mal classifiées
- **Problème** : cas où les classes ne sont **pas équilibrées**
 - Exemple** : détection de fraude
 - 99% des observations ne sont pas des fraudes
 - Un modèle qui prédit toujours « non » a un pourcentage d'erreur de 1%.
- **Matrice de confusion** (confusion matrix)

		Classe réelle	
		0	1
Classe prédite	0	Vrais Négatifs (TN)	Faux Négatifs (FN)
	1	Faux Positifs (FP)	Vrais Positifs (TP)

1.4 Évaluation d'un modèle (classification)

- **Pourcentage d'erreur** : proportion d'observations mal classifiées
- **Problème** : cas où les classes ne sont **pas équilibrées**
 - Exemple** : détection de fraude
 - 99% des observations ne sont pas des fraudes
 - Un modèle qui prédit toujours « non » a un pourcentage d'erreur de 1%.
- **Matrice de confusion** (confusion matrix)

		True class	
		0	1
Predicted class	0	True Negatives (TN)	False Negatives (FN)
	1	False Positives (FP)	True Positives (TP)

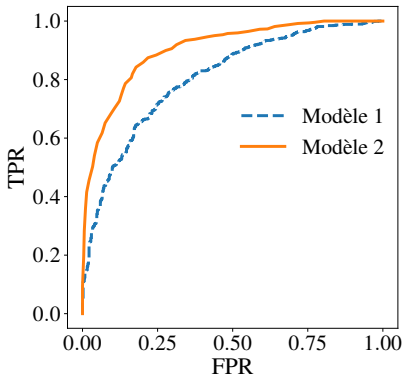
1.4 Évaluation d'un modèle (classification)

		Classe réelle		
		0	1	
Classe prédite	0	Vrais Négatifs (TN)	Faux Négatifs (FN)	TN+FN
	1	Faux Positifs (FP)	Vrais Positifs (TP)	TP+FP
		TN + FP	TP + FN	

- Sensibilité (sensitivity) = rappel (recall) = taux de vrais positifs = $\frac{TP}{TP + FN}$
- Spécificité (specificity) = taux de vrais négatifs (TN rate) = $\frac{TN}{TN + FP}$
- Précision (precision) = $\frac{TP}{TP + FP}$
- Précision (accuracy) = $\frac{TP + TN}{TP + FP + TN + FN}$
- Score F (F-score) = moyenne harmonique (précision, rappel) = $\frac{2 TP}{TP + FP + TP + FN}$

Évaluation d'un modèle (courbe ROC)

- Receiver-Operator Characteristic curve



- Évaluation numérique : aire sous la courbe ROC (AUC, AUROC)

1.5 Évaluation d'un modèle (régression)

- **Compter** le nombre d'erreurs n'a pas de sens

1.5 Évaluation d'un modèle (régression)

- **Compter** le nombre d'erreurs n'a pas de sens
- Somme des carrés des erreurs : $RSS = \sum_{i=1}^n (y_i - f(\vec{x}_i))^2$
- Racine de l'erreur quadratique moyenne (Root Mean Squared Error) :

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - f(\vec{x}_i))^2}$$

- Erreur carrée relative : $RSE = \frac{\sum_{i=1}^n (y_i - f(\vec{x}_i))^2}{\sum_{i=1}^n (y_i - \bar{y})^2} \quad \bar{y} = \sum_{l=1}^n y_l$

- Coefficient de détermination :

$$R^2 = 1 - RSE = \frac{\sum_{i=1}^n (y_i - \bar{y}) (f(\vec{x}_i) - \overline{f(\vec{x})})}{\sqrt{\sum_{i=1}^n (y_i - \bar{y})^2} \sqrt{\sum_{i=1}^n (f(\vec{x}_i) - \overline{f(\vec{x})})^2}}$$

1.6 Sélection de modèle

- Comment déterminer **le meilleur modèle** parmi ceux appris :
 - avec différents algorithmes d'apprentissage ;
 - avec différentes valeurs d'hyperparamètre(s) pour le même algorithme ?

1.6 Sélection de modèle

- Comment déterminer **le meilleur modèle** parmi ceux appris :
 - avec différents algorithmes d'apprentissage ;
 - avec différentes valeurs d'hyperparamètre(s) pour le même algorithme ?
- **Idée** : sélectionner celui qui a la **meilleure performance sur le jeu de test**.

1.6 Sélection de modèle

- Comment déterminer **le meilleur modèle** parmi ceux appris :
 - avec différents algorithmes d'apprentissage ;
 - avec différentes valeurs d'hyperparamètre(s) pour le même algorithme ?
- **Idée** : sélectionner celui qui a la **meilleure performance sur le jeu de test**.
- **Problème** : on ne peut plus déterminer l'**erreur de généralisation** car les données de test ont déjà servi.

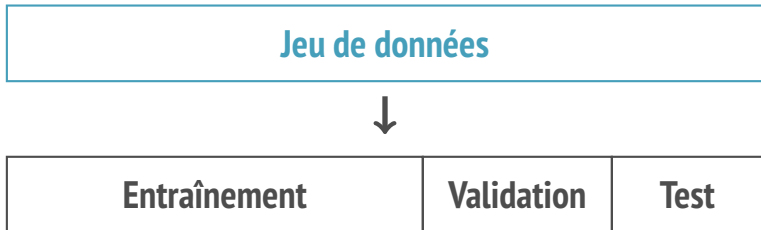
1.6 Sélection de modèle

- Comment déterminer **le meilleur modèle** parmi ceux appris :
 - avec différents algorithmes d'apprentissage ;
 - avec différentes valeurs d'hyperparamètre(s) pour le même algorithme ?
- **Idée** : sélectionner celui qui a la **meilleure performance sur le jeu de test**.
- **Problème** : on ne peut plus déterminer l'**erreur de généralisation** car les données de test ont déjà servi.
- On sépare les données en 3 jeux : apprentissage, **validation** et test.

Jeu de données

1.6 Sélection de modèle

- Comment déterminer **le meilleur modèle** parmi ceux appris :
 - avec différents algorithmes d'apprentissage ;
 - avec différentes valeurs d'hyperparamètre(s) pour le même algorithme ?
- **Idee** : sélectionner celui qui a la **meilleure performance sur le jeu de test**.
- **Problème** : on ne peut plus déterminer l'**erreur de généralisation** car les données de test ont déjà servi.
- On sépare les données en 3 jeux : apprentissage, **validation** et test.



Jeu de validation

Entraînement	Validation	Test
--------------	------------	------

- Le **jeu d'entraînement** sert à apprendre **chacun des** modèles
- Le **jeu de validation** sert à **sélectionner** le meilleur modèle
- Le **jeu de test** sert à estimer l'erreur de généralisation **du modèle sélectionné**

Jeu de validation

Entraînement	Validation	Test
--------------	------------	------

- Le **jeu d'entraînement** sert à apprendre **chacun des** modèles
- Le **jeu de validation** sert à **sélectionner** le meilleur modèle
- Le **jeu de test** sert à estimer l'erreur de généralisation **du modèle sélectionné**
- **Problèmes :**
 - On utilise seulement une fraction des données pour l'entraînement
 - Possibilité que le modèle sélectionné ait une meilleure performance sur ce jeu de validation précis à cause d'un artefact.

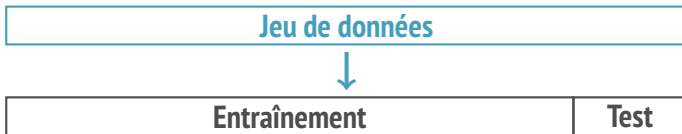
Validation croisée (sélection de modèle)

- On peut aussi utiliser la **validation croisée** pour la **sélection de modèle**
- On sépare le jeu d'entraînement en K **blocs** (ou folds)
- Pour chaque algorithme ou valeur d'hyperparamètre à évaluer :
 - On utilise tour à tour chacun des blocs comme **jeu de validation** et l'union des autres comme **jeu d'entraînement**
- ⇒ on obtient K scores de performance → **performance moyenne**
- On sélectionne l'algorithme ou la valeur d'hyperparamètre qui donne la meilleure performance moyenne
- On réentraîne sur le jeu d'entraînement l'algorithme sélectionné
- On estime l'erreur de généralisation en évaluant sur le jeu de test le modèle ainsi obtenu

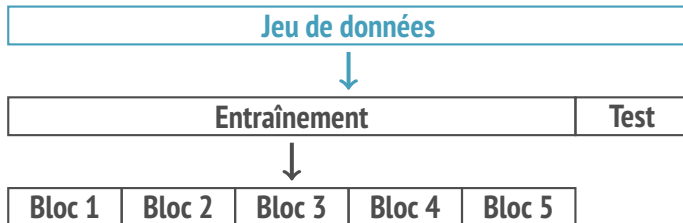
Validation croisée (sélection de modèle)

Jeu de données

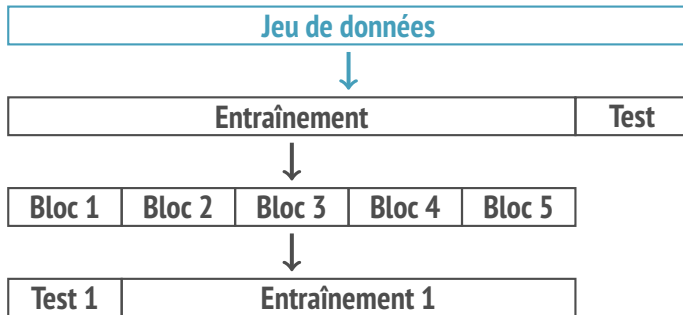
Validation croisée (sélection de modèle)



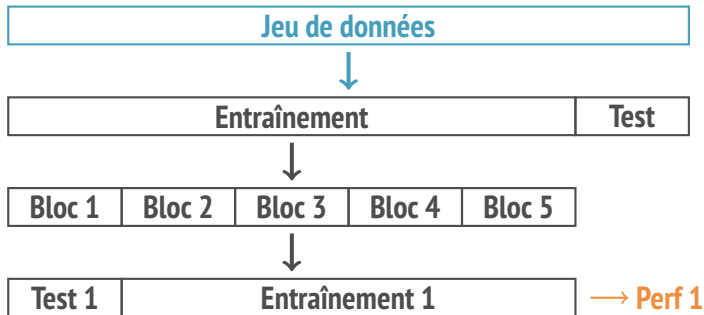
Validation croisée (sélection de modèle)



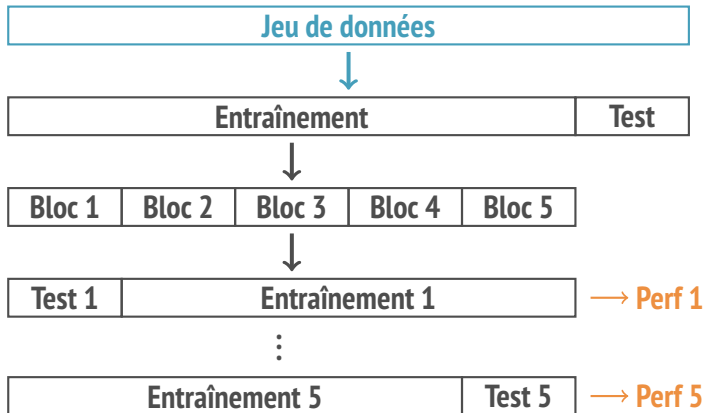
Validation croisée (sélection de modèle)



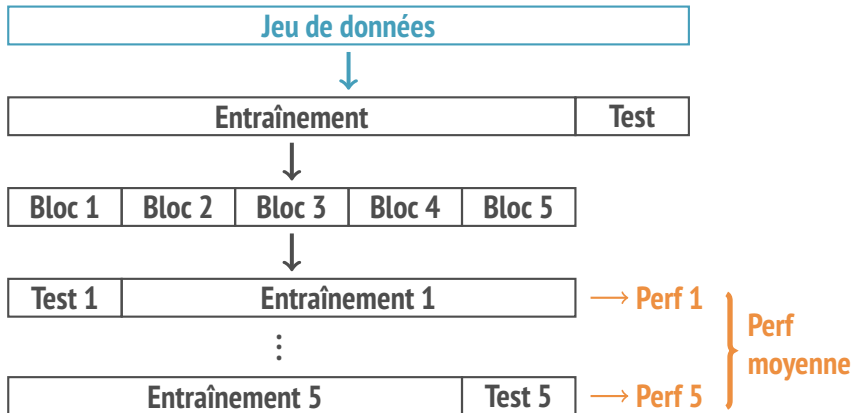
Validation croisée (sélection de modèle)



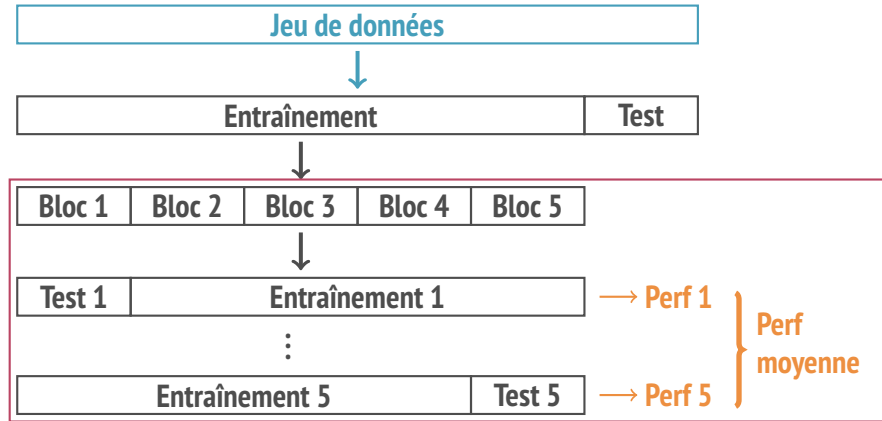
Validation croisée (sélection de modèle)



Validation croisée (sélection de modèle)

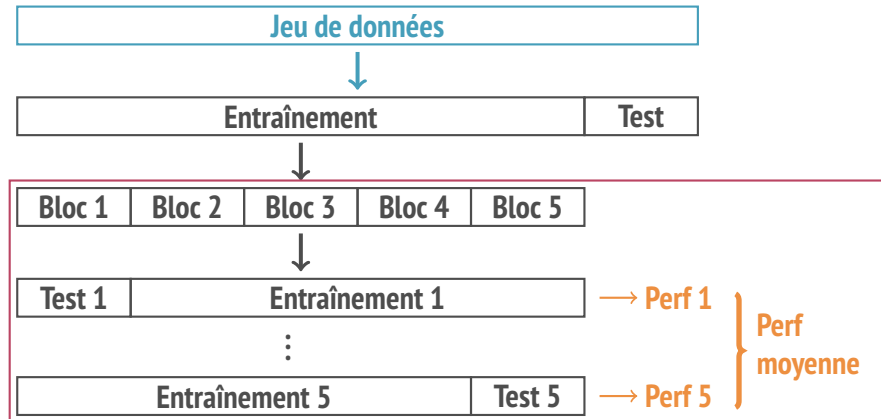


Validation croisée (sélection de modèle)



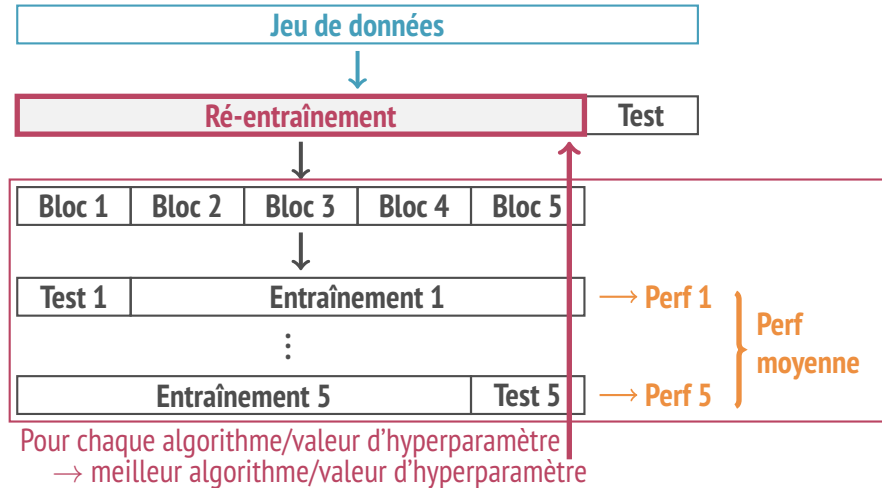
Pour chaque algorithme/valeur d'hyperparamètre

Validation croisée (sélection de modèle)

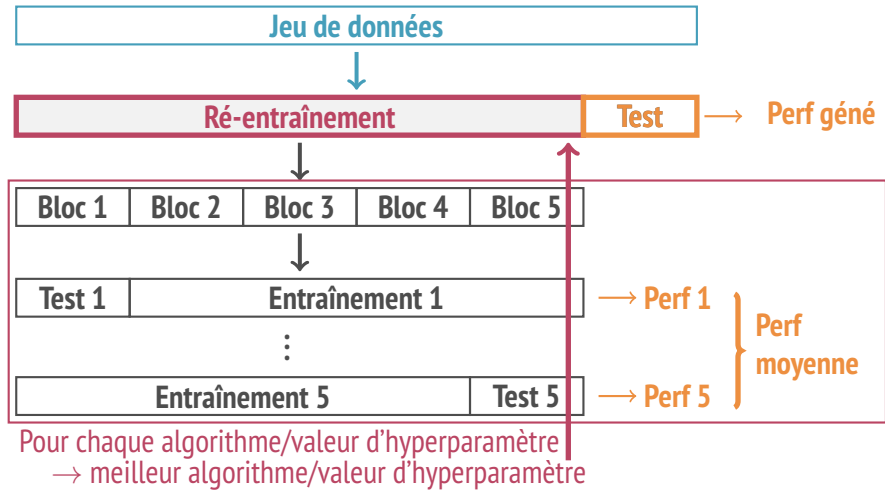


Pour chaque algorithme/valeur d'hyperparamètre
→ meilleur algorithme/valeur d'hyperparamètre

Validation croisée (sélection de modèle)



Validation croisée (sélection de modèle)

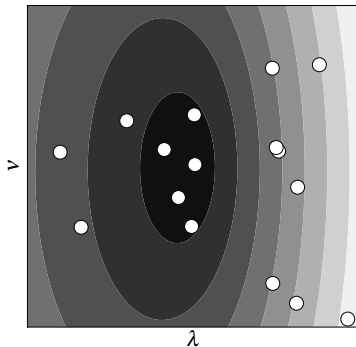
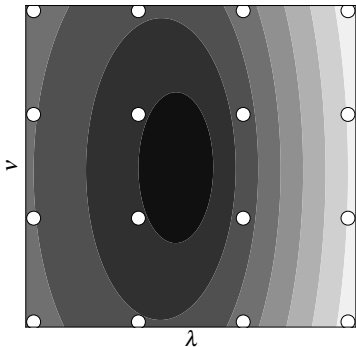


Validation croisée imbriquée

- Utiliser une validation croisée :
 - pour **la sélection de modèle** (boucle interne)
 - et pour **l'évaluation** du modèle sélectionné (boucle externe)
- Peut devenir **coûteux en calculs** :
 - **Exemple** : si $K = 5$ pour la boucle interne, $K=10$ pour la boucle externe, et on a 10 valeurs d'hyperparamètre à évaluer : 500 modèles à entraîner.
 - Cependant, facile à paralléliser.

Plusieurs hyperparamètres

- **Recherche par grille** (grid search)
- **Recherche gloutonne** : optimization successive (greedy search)
- **Recherche aléatoire** (random search)



2. Régularisation

2.1 Régularisation

- Par principe, la **minimisation du risque empirique** conduit facilement au **surapprentissage**
- La **régularisation** consiste à **contraindre** le problème pour **limiter la complexité** du modèle
 - **Complexité** : notion théorique, par ex. Vapnik–Chervonenkis
 - Intuitivement : « flexibilité » des modèles que l'on peut apprendre
 - Proxys : nombre de paramètres ; amplitude possible de ces paramètres

$$\hat{f} = \arg \min_{f \in \mathcal{F}} \frac{1}{n} \sum_{i=1}^n L(y_i, f(\vec{x}_i)) + \lambda \Omega(f) \quad \lambda > 0$$

Coefficient de régularisation

$$\hat{f} = \arg \min_{f \in \mathcal{F}} \frac{1}{n} \sum_{i=1}^n L(y_i, f(\vec{x}_i)) + \lambda \Omega(f) \quad \lambda > 0$$

- **Coefficient de régularisation** $\lambda > 0$: **hyperparamètre** qui gouverne l'importance relative entre la minimisation du risque empirique et la complexité $\Omega(f)$ du modèle.
- $\lambda = 0$:
- $\lambda \rightarrow +\infty$:
- Déterminer λ :

2.2 Régularisation ridge

- Exemple 1 : **Régularisation ridge** (ou ℓ_2 , ou **weight decay**)
- Pour un modèle paramétrique de paramètres $\theta_1, \theta_2, \dots, \theta_d$,
$$\Omega(\vec{\theta}) = \left\| \vec{\theta} \right\|_2^2 = \sum_{j=1}^d \theta_j^2$$
 contrôle **l'amplitude des paramètres**

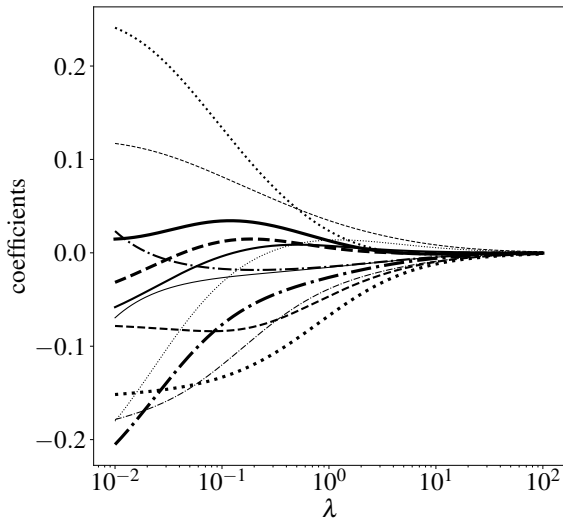
2.2 Régularisation ridge

- Exemple 1 : **Régularisation ridge** (ou ℓ_2 , ou **weight decay**)
 - Pour un modèle paramétrique de paramètres $\theta_1, \theta_2, \dots, \theta_d$,
 $\Omega(\vec{\theta}) = \left\| \vec{\theta} \right\|_2^2 = \sum_{j=1}^d \theta_j^2$ contrôle **l'amplitude des paramètres**
- **Régression ridge :**

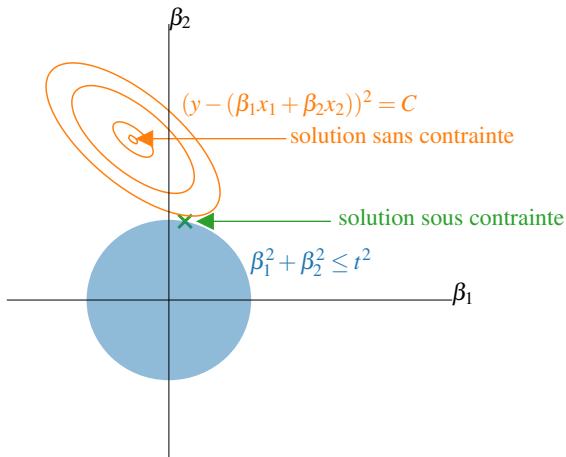
$$\arg \min_{\vec{\beta} \in \mathbb{R}^{p+1}} \frac{1}{n} \left(\vec{y} - X\vec{\beta} \right)^\top \left(\vec{y} - X\vec{\beta} \right) + \lambda \left\| \vec{\beta} \right\|_2^2$$

Admet toujours une unique solution $\vec{\beta}^* = (X^\top X + \lambda I_p)^{-1} X^\top \vec{y}$
car ajouter une matrice diagonale à coefficients strictement positifs à $X^\top X$ la rend inversible

Chemin de régularisation (ridge)



Interprétation géométrique



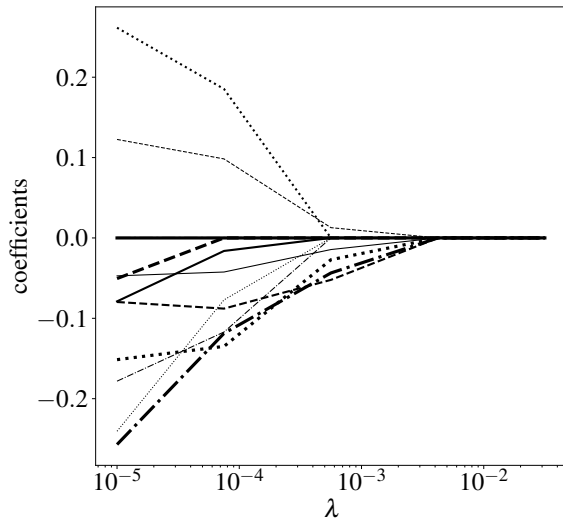
2.3 Lasso

- Exemple 2 : **Régularisation ℓ_1**
- Pour un modèle paramétrique de paramètres $\theta_1, \theta_2, \dots, \theta_d$,
 $\Omega(\vec{\theta}) = \|\vec{\theta}\|_1 = \sum_{j=1}^d |\theta_j|$ contrôle **le nombre de paramètres non nuls** $\rightarrow$ **parcimonie** (sparsity)

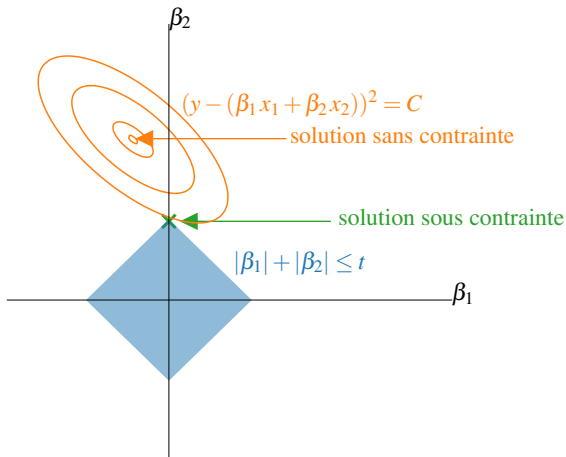
2.3 Lasso

- Exemple 2 : **Régularisation ℓ_1**
 - Pour un modèle paramétrique de paramètres $\theta_1, \theta_2, \dots, \theta_d$,
 $\Omega(\vec{\theta}) = \left\| \vec{\theta} \right\|_1 = \sum_{j=1}^d |\theta_j|$ contrôle **le nombre de paramètres non nuls** $\rightarrow$ **parcimonie** (sparsity)
 - **Lasso** (Least Absolute Sparse Selection Operator) :
$$\arg \min_{\vec{\beta} \in \mathbb{R}^{p+1}} \frac{1}{n} \left(\vec{y} - X\vec{\beta} \right)^\top \left(\vec{y} - X\vec{\beta} \right) + \lambda \left\| \vec{\beta} \right\|_1$$
 - Résolu par l'algorithme du gradient.

Chemin de régularisation



Interprétation géométrique



Elastic Net

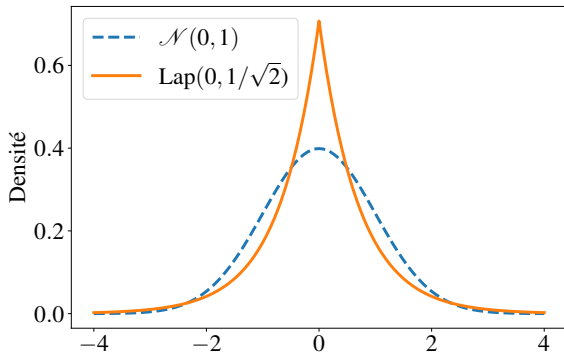
- Variables **corrélées** : le Lasso sélectionne l'une d'entre elles aléatoirement

⇒ **instabilité**

- **Elastic Net**

$$\Omega(\vec{\theta}) = \alpha \left\| \vec{\theta} \right\|_2^2 + (1 - \alpha) \left\| \vec{\theta} \right\|_1 \quad 0 < \alpha < 1$$

Estimation par maximum a posteriori



Régression polynomiale

- Comment apprendre un modèle **non-linéaire** avec ces éléments ?

Références / page pub

- **Introduction au Machine Learning**, Chloé-Agathe Azencott, Dunod InfoSup
Version électronique (PDF) sans exercices disponible gratuitement sur <http://cazencott.info> [lien cliquable]
(nouvelle édition en préparation)
- **Le Machine Learning avec scikit-learn**, Aurélien Géron, Dunod InfoSup
- Textes et vidéos sur **OpenClassrooms** :
Textes accessibles librement ; vidéos accessibles avec un compte gratuit.
 - Thématique : Data [lien cliquable]
 - Objectif IA (grand public) [lien cliquable]

Ressources complémentaires

- **Sources de jeux de données**
 - Kaggle [\[lien cliquable\]](#)
 - Le module datasets de scikit-learn [\[lien cliquable\]](#)
 - Le UCI Machine Learning Repository [\[lien cliquable\]](#)
- **MOOC scikit-learn** [\[lien cliquable\]](#)